Communication & Integration Patterns

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

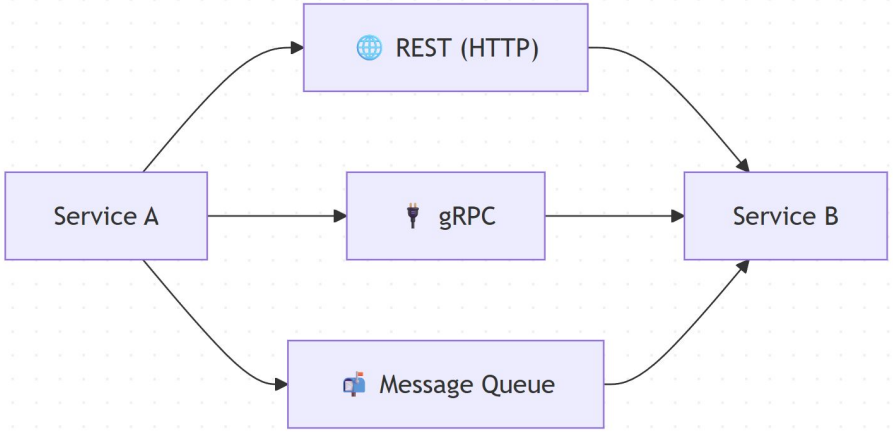
Communication Patterns: Sync vs Async

Communication & Integration Patterns

Mastering Software Architecture Patterns and System Design

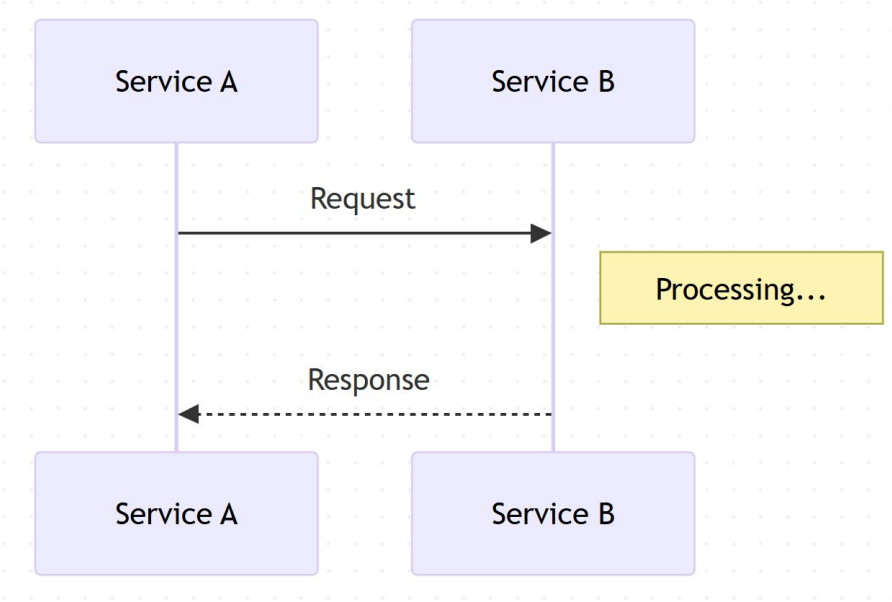
Why Communication Patterns Matter

- Communication = glue between distributed services
- Poor choices = cascading failures, latency issues, or poor developer experience
- Impacts system design, scalability, observability



Synchronous Communication – Core Concepts

- Caller waits for response (blocking)
- Typically HTTP (REST, gRPC)
- Easier to reason about and debug
- Tight coupling in time (if callee is slow, caller waits)

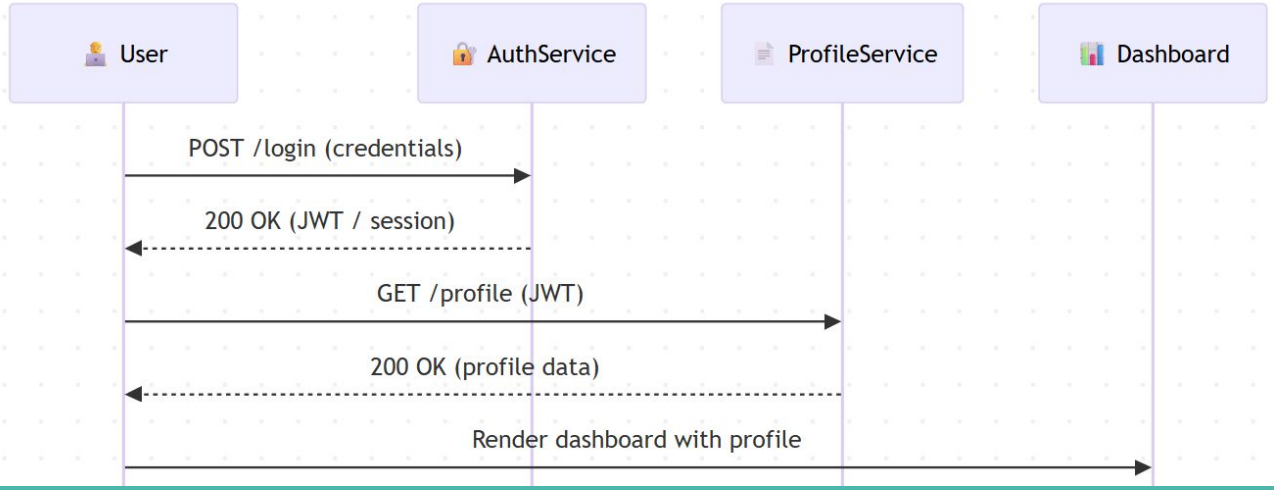


Common Sync Protocols

- REST over HTTP: Most common, human-readable, flexible
- gRPC: High performance, binary protocol, strongly typed (used in internal service-to-service)
- GraphQL: (optional mention) Client-driven queries, still synchronous

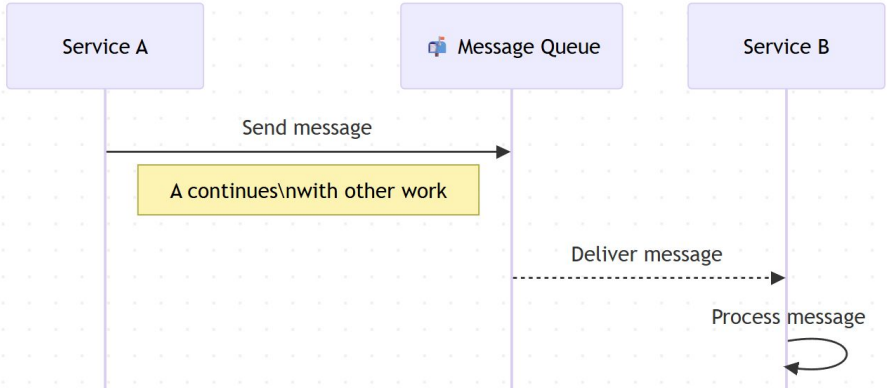
When to Use Synchronous Communication

- When immediate result is needed (e.g., payment auth)
- For user-facing interactions
- For tightly-coupled flows (login → token → profile)
- When simplicity is more important than flexibility



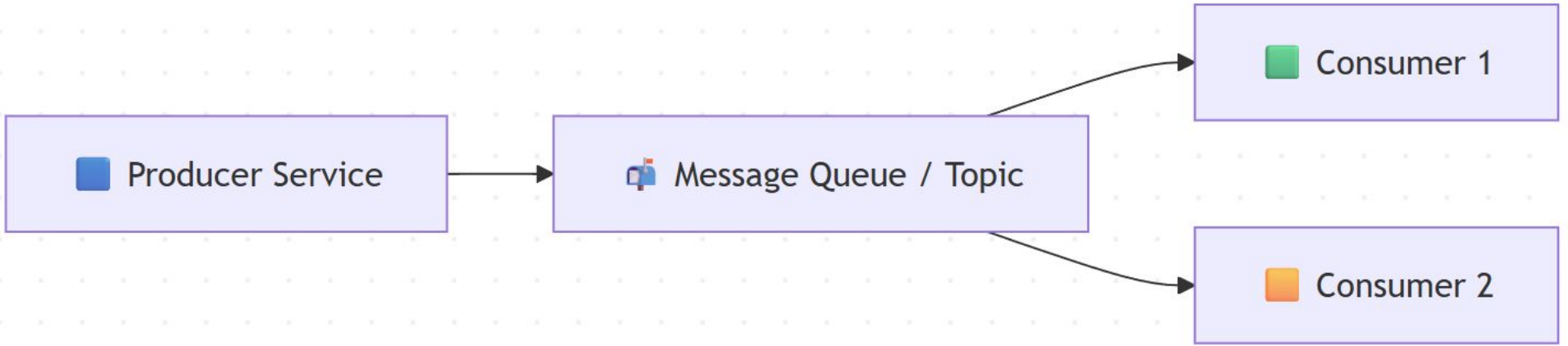
Asynchronous Communication – Core Concepts

- Caller does not wait (fire-and-forget or eventual response)
- Useful when tasks are long-running or not immediately required
- Increases resilience, decouples timing



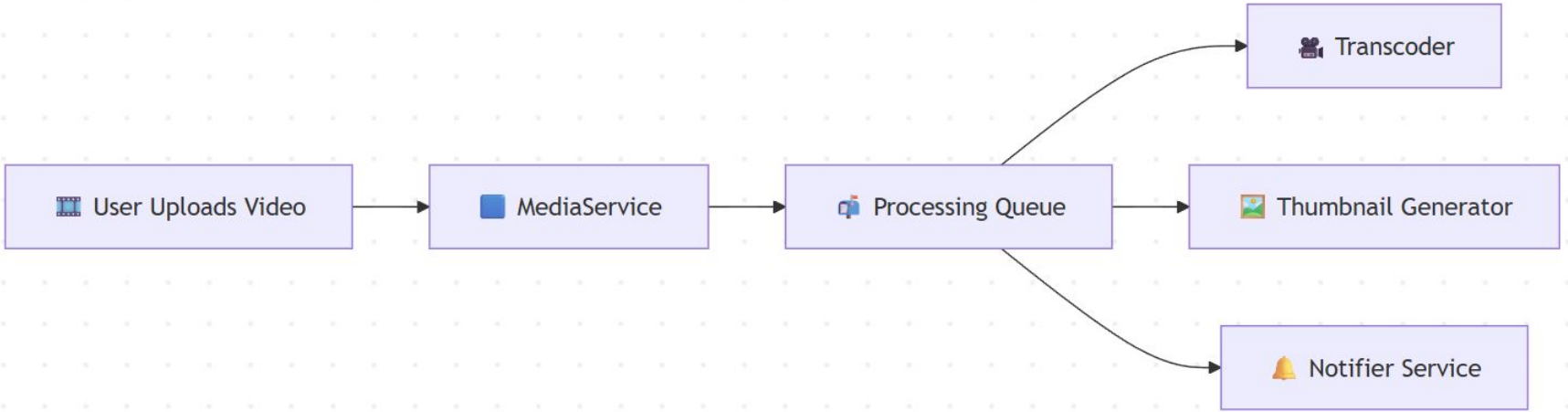
Asynchronous Tools & Protocols

- Message Queues (Kafka, RabbitMQ, SQS)
- Pub/Sub systems (Google Pub/Sub, AWS SNS)
- Webhooks (event-based HTTP callbacks)
- Event Buses (e.g., EventBridge, NATS)



When to Use Asynchronous Communication

- Background processing (e.g., media uploads, email notifications)
- High-throughput data ingestion
- Retryable operations, fault tolerance
- Loosely-coupled services with domain events

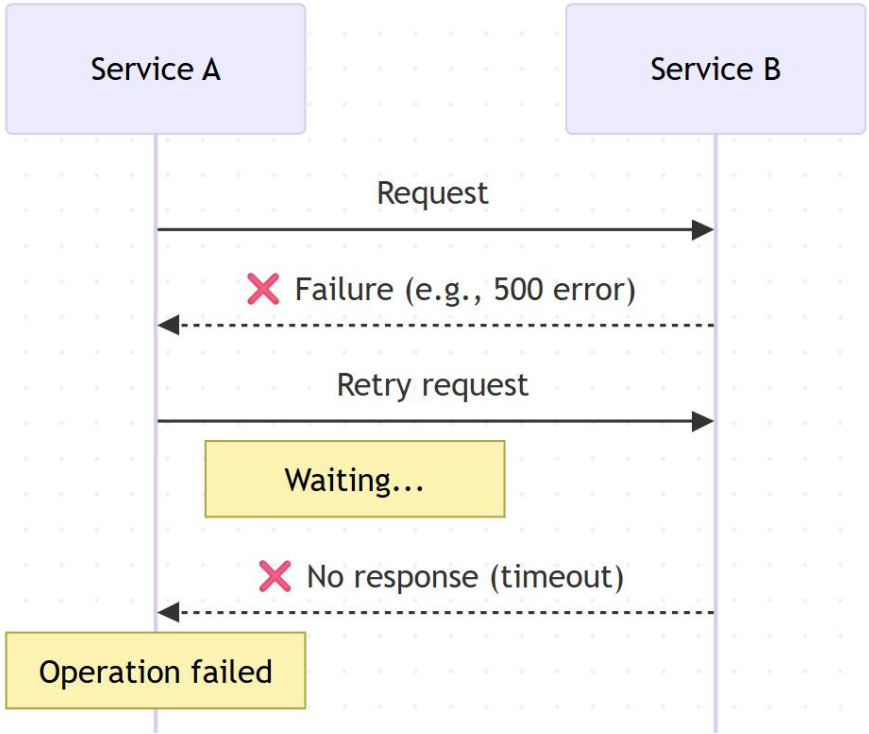


Sync vs Async – Trade-offs

- 🔁 Synchronous Communication
  - Caller waits for the response (blocking)
  - Simpler to implement and debug
  - Feels more intuitive in request-response flows
  - Tightly coupled in time (both services must be up)
  - Higher risk of cascading failures
  - Ideal for user-facing flows where immediate feedback is needed
- 🔄 Asynchronous Communication
  - Caller does not wait (non-blocking)
  - Enables loose coupling and fault isolation
  - Supports retry, buffering, and durability
  - Harder to debug (requires tracing/log correlation)
  - Introduces eventual consistency
  - Ideal for background tasks, high-throughput, or resilient systems

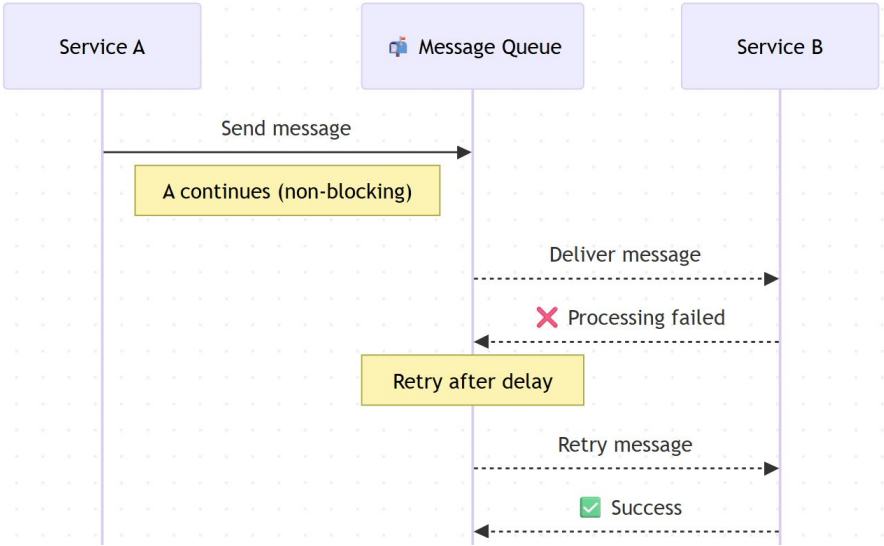
Visualizing Failure in Synchronous Systems

- Service B failure causes Service A to fail or timeout
- Often leads to retry storms or cascading failures
- Requires retries, timeouts, fallbacks



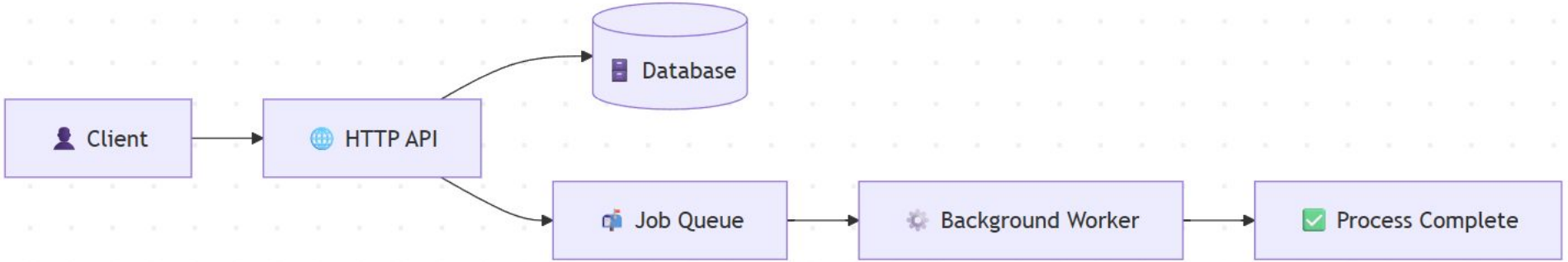
Async Resilience Pattern

- Queues store events safely
- Consumers can retry independently
- Caller unaffected by downstream failures



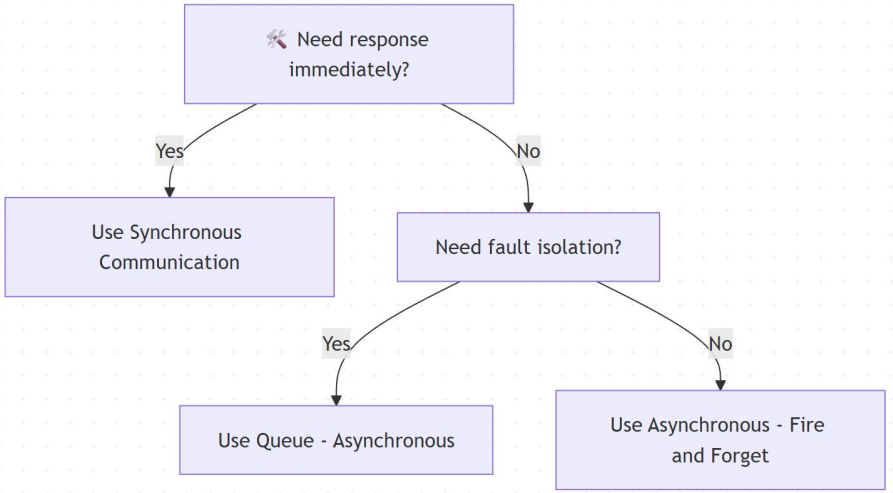
Mixed Architectures – Best of Both Worlds

- Many systems use hybrid patterns
- Example: API triggers async pipeline
- Sync for request acknowledgement, async for heavy lifting



Impact on System Design

- Sync = simpler design, tighter feedback
- Async = resilient, scalable, harder to debug
- Choose based on: latency needs, tolerance to delay, failure recovery model



Summary – Sync vs Async

- Synchronous: clean, simple, user-facing
- Asynchronous: scalable, resilient, background workflows
- Modern systems use both – designed intentionally
- Think in terms of: timing, decoupling, and failure boundaries
- What's next:
  - API Gateway & Backend for Frontend (BFF)

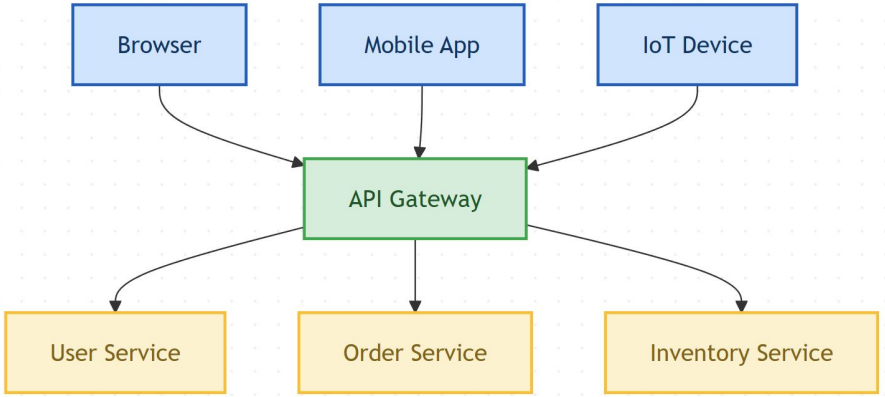
API Gateway & Backend for Frontend (BFF)

Communication & Integration Patterns

Mastering Software Architecture Patterns and System Design

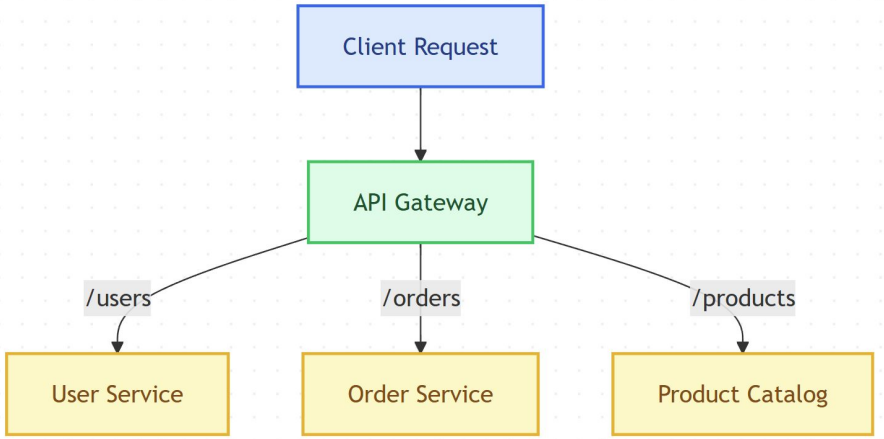
Why Do We Need an API Gateway?

- Centralized entry point to your system.
- Reduces client complexity.
- Handles security, rate-limiting, logging, etc.
- Allows backend service evolution without breaking clients.



Gateway Routing

- Routes traffic based on paths (e.g., /users, /orders).
- Can do host-based routing (api.mobile.com vs api.web.com).
- Reduces hardcoded URLs on the client.



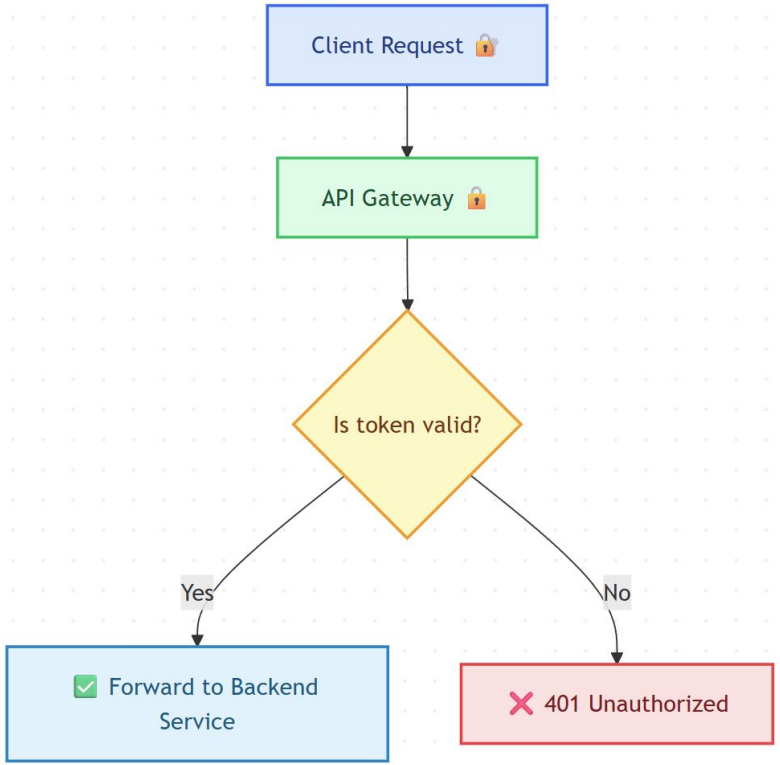
Request/Response Transformation

- Modify headers, format, structure.
- Useful when backend and frontend follow different conventions.
- Enables backward compatibility.



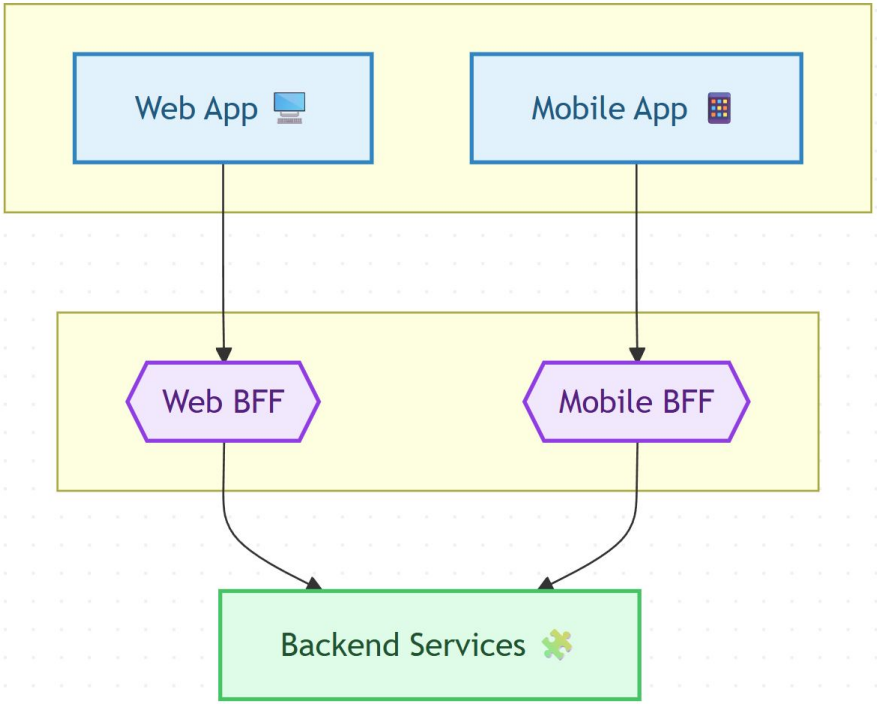
Authentication & Authorization at the Gateway

- Validates tokens (OAuth2, JWT).
- Verifies API keys and roles.
- Can block unauthorized requests before reaching backend.



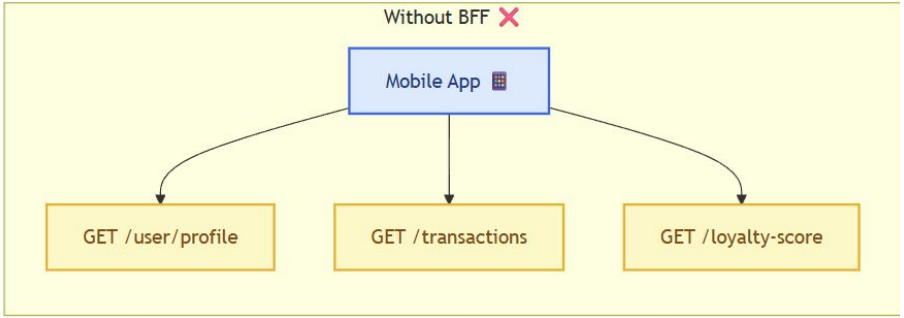
What is Backend for Frontend (BFF)?

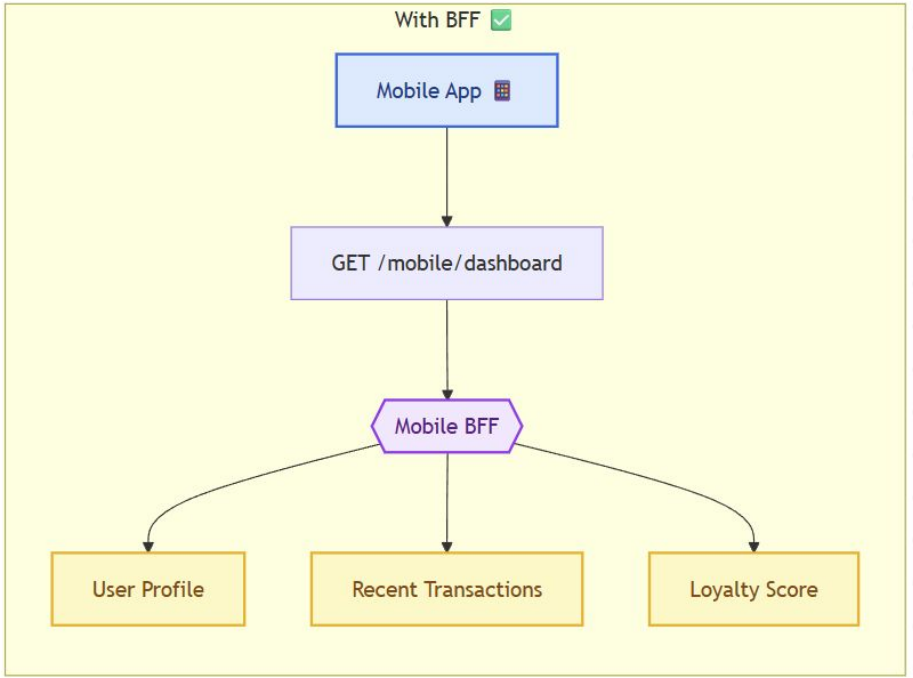
- Each client (Web, Mobile) gets its own tailored backend.
- BFF adapts backend APIs to frontend needs.
- Frontend teams can own their own BFF.
- Example:
  - Netflix Mobile BFF sends compressed, personalized data.
  - Web BFF sends large, rich payloads.



Why BFF Instead of Generic Gateway?

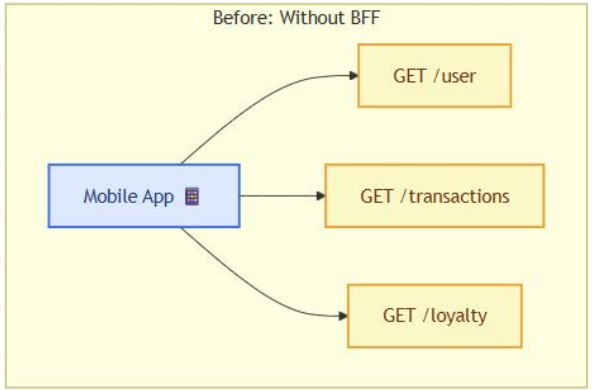
- Prevents frontend complexity.
- Tailored responses: mobile needs less data, different formats.
- Enables frontend-specific orchestration.

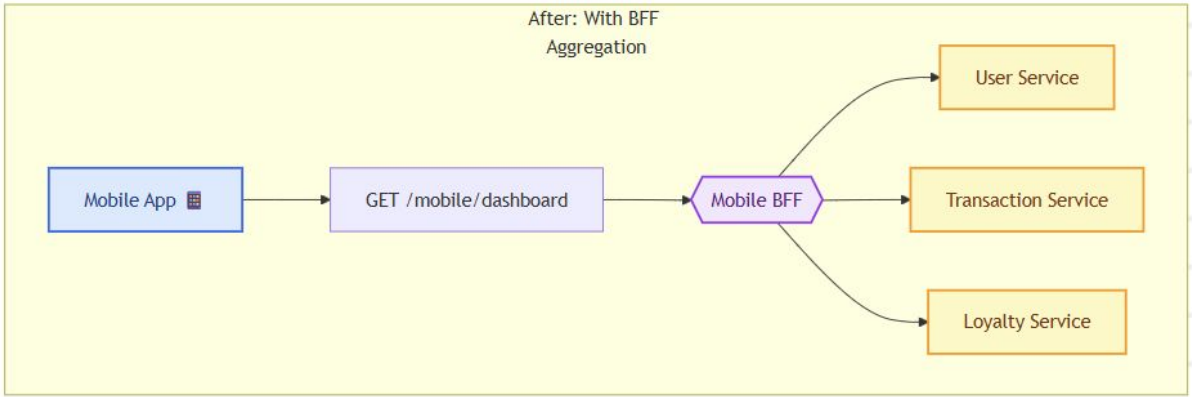




Aggregation & Orchestration Example

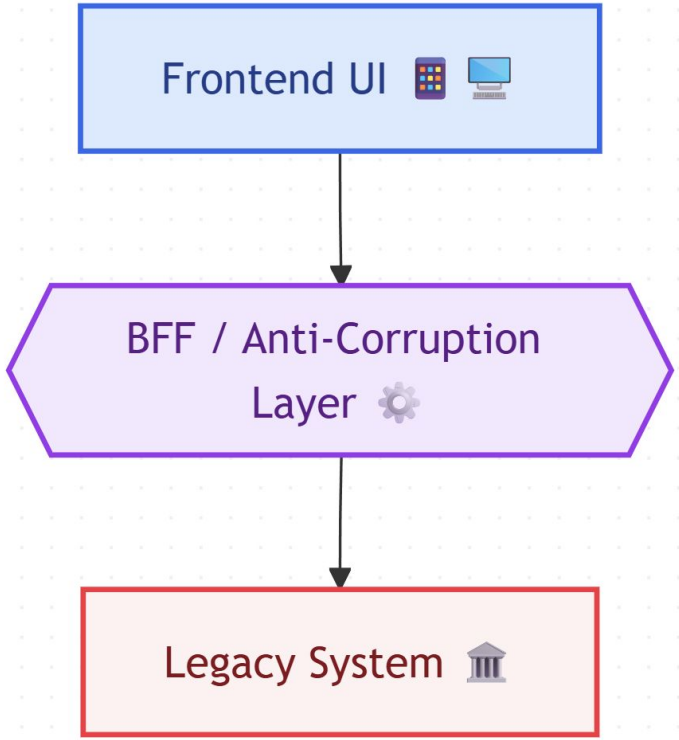
- Mobile app shows a dashboard needing:
  - User Profile
  - Recent Transactions
  - Loyalty Score
- Points:
  - BFF fetches all data from services and aggregates.





Anti-Corruption Layer via BFF

- Prevent frontend from dealing with legacy quirks.
- BFF acts as a “translator” or façade.

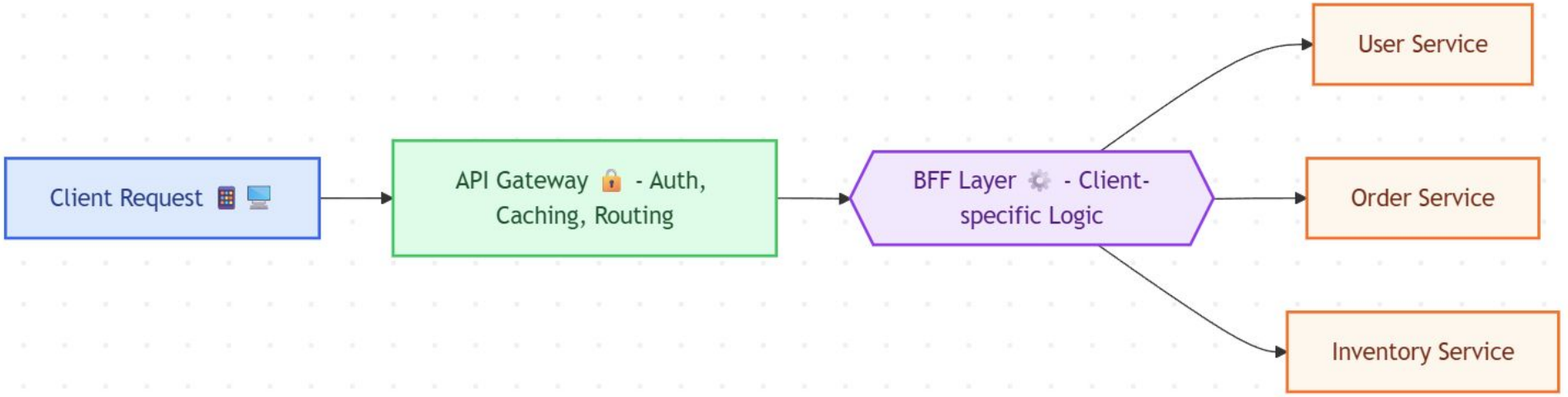


API Gateway vs BFF – When to Use What

- ✅ Use an API Gateway when:
  - You need a centralized entry point for all clients.
  - You want to enforce authentication, rate limiting, logging, or global caching.
  - You aim to standardize access to backend microservices.
  - You have multiple APIs/services and want to hide internal structure.
  - Your team prefers shared infrastructure ownership (DevOps or platform team).
- ✅ Use a BFF when:
  - Each client (web, mobile, desktop) has different API needs.
  - You want to custom-tailor responses to reduce bandwidth or optimize UX.
  - You want to delegate orchestration or aggregation logic to the frontend team.
  - You're dealing with legacy systems and want to isolate them via anti-corruption layers.
  - You want frontend teams to move independently from backend teams.
- 🔁 Use Both Together when:
  - API Gateway handles cross-cutting concerns (auth, throttling, logging).
  - BFFs handle client-specific logic, composition, or abstraction of legacy systems.
  - You want to balance infrastructure control and frontend flexibility.

API Gateway + BFF Together

- Gateway handles global concerns (auth, logging).
- Gateway forwards to BFFs.
- BFFs handle orchestration, formatting.



Challenges & Best Practices

- Avoid bloated BFFs.
- Ensure versioning and testing.
- Secure internal communication (mTLS, service mesh).
- Gateway ≠ replacement for service design.

Summary - API Gateway & Backend for Frontend (BFF)

- API Gateway = security, standardization
- BFF = flexibility, frontend optimization
- Together = power and adaptability
- What's next:
  - Messaging and Event Brokers

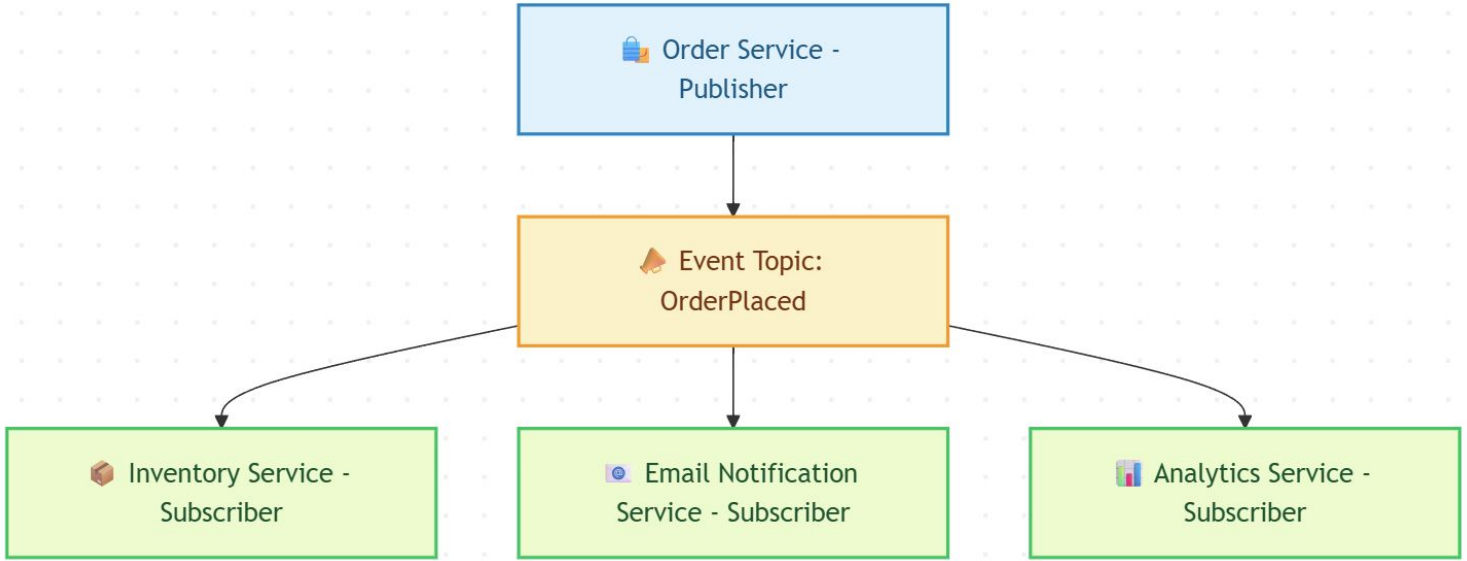
Messaging and Event Brokers

Communication & Integration Patterns

Mastering Software Architecture Patterns and System Design

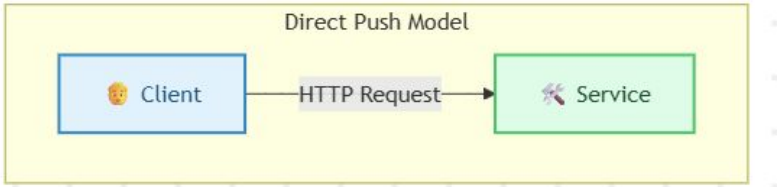
Why Messaging?

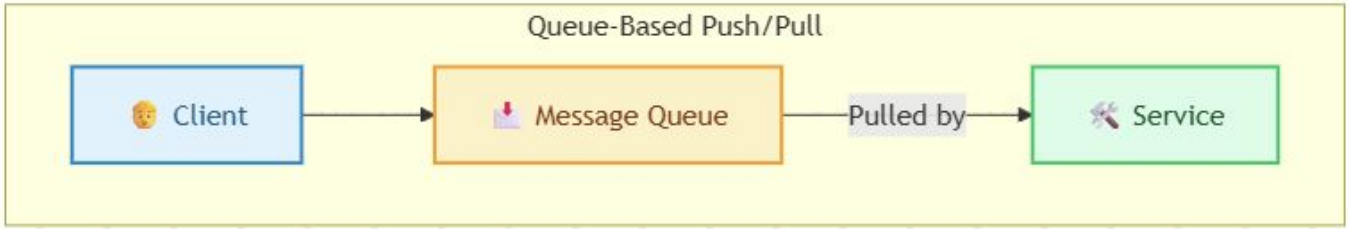
- Direct calls (like REST) tightly couple services.
- Messaging enables asynchronous, resilient, scalable communication.
- Helps services operate independently.



Push vs. Pull Communication

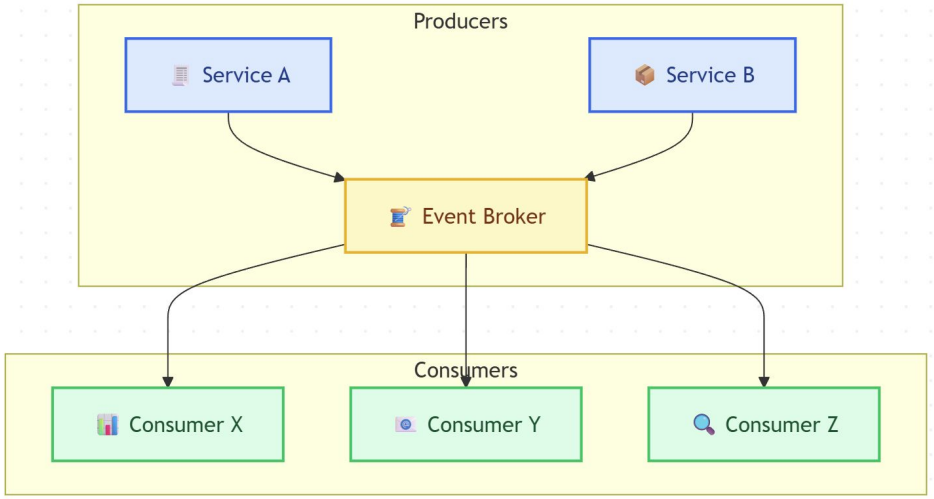
- In REST: Service A calls B directly (push).
- In Messaging: A pushes to queue, B pulls when ready.
- Enables load smoothing and better fault tolerance.





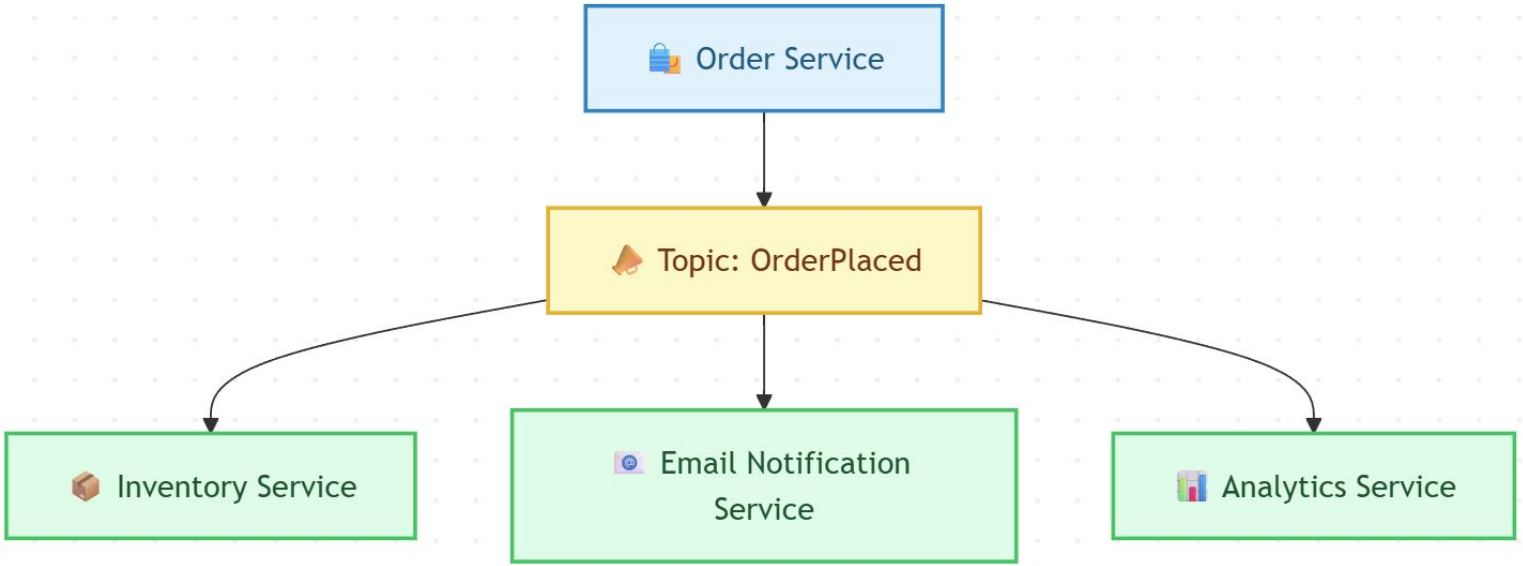
What is an Event Broker?

- A middleware system that receives, stores, and forwards messages or events.
- Allows many producers and many consumers.
- Central to event-driven architectures.



Publish-Subscribe Model

- Producers publish events.
- Subscribers receive events matching their interest.
- Fully decoupled: producers don’t know who’s listening.



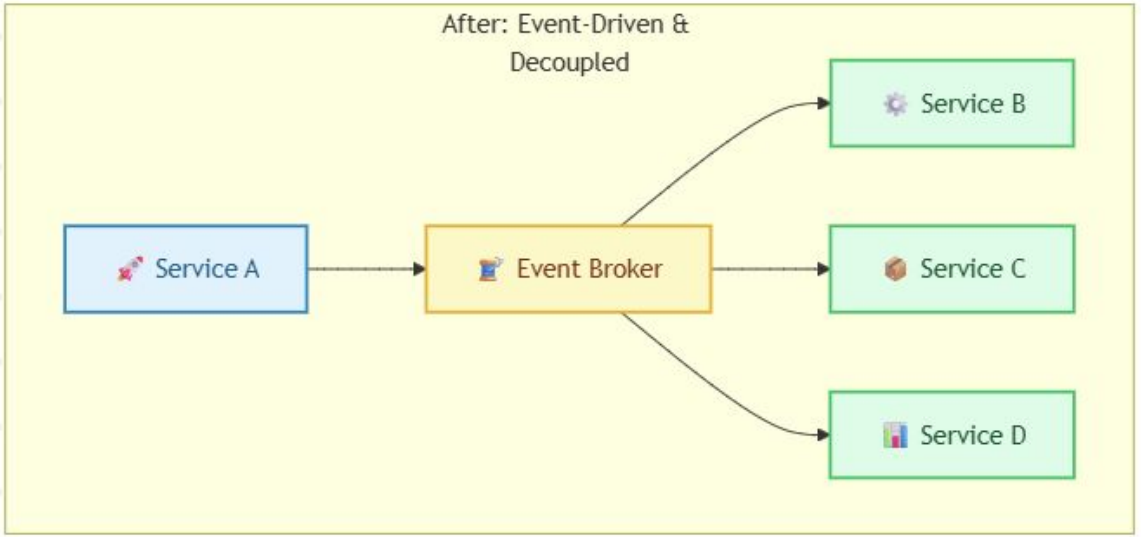
Queues vs. Streams

- 🟦 Message Queues (e.g., RabbitMQ, SQS):
  - Point-to-point delivery — each message goes to one consumer.
  - Messages are removed once delivered and acknowledged.
  - Great for task distribution (e.g., email jobs, background tasks).
  - Supports routing, acknowledgement, dead-letter queues.
- 🟧 Event Streams (e.g., Kafka, Kinesis):
  - Multiple consumers can read the same message independently.
  - Messages are retained for a set time (e.g., 7 days).
  - Ideal for analytics, event sourcing, stream processing.
  - Consumers manage their own position (offsets) in the stream.

Decoupled Architecture with Events

- Event brokers allow independent scaling of services.
- Services don’t wait on each other.
- New consumers can be added without changing producers.





When to Use Messaging?

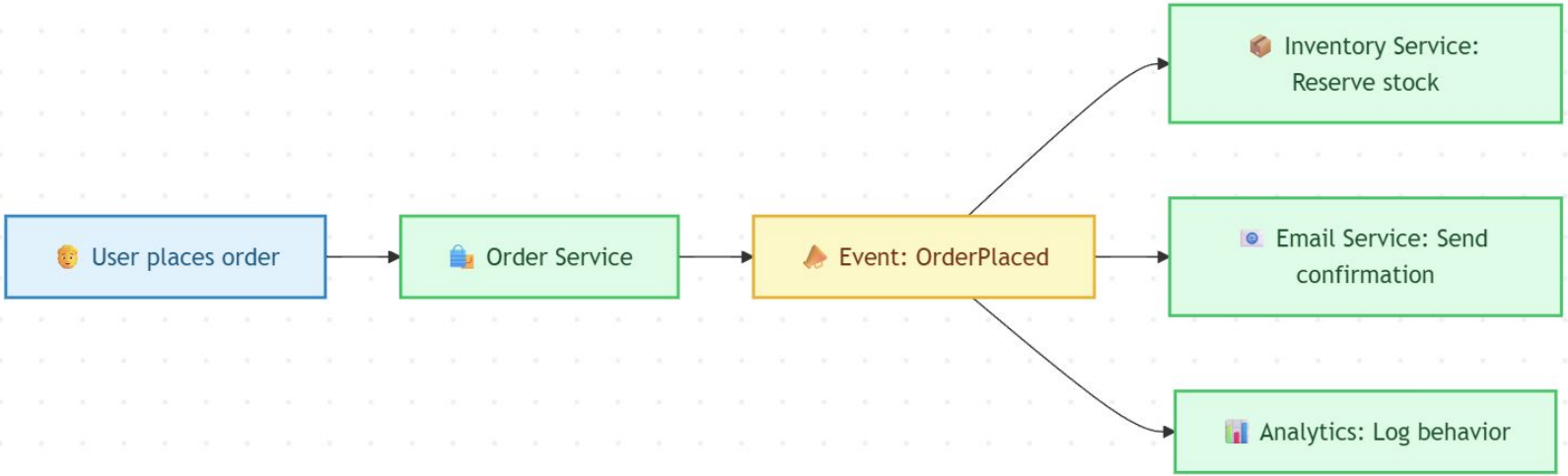
- ✅ Great for:
  - Asynchronous processing (e.g., email, billing, logging)
  - High-throughput pipelines
  - Decoupling microservices
- ❌ Avoid if:
  - You need immediate response
  - The use case is simple (adds infra complexity)

Common Pitfalls

- Messages lost if not acknowledged (check durability settings).
- Consumers may get out of sync (offset management).
- Harder to debug than direct APIs.
- Eventual consistency must be accepted.

Example: Order Placed Event Flow

- User places order → OrderPlaced event published
- Inventory reserves stock
- Email service sends confirmation
- Analytics logs user behavior



Summary – Messaging and Event Brokers

- Messaging helps decouple systems and improve reliability.
- Queues are for tasks, streams are for events.
- RabbitMQ and Kafka are popular tools.
- Great for scaling and flexibility — but use carefully.
- What's next:
  - Resilience Patterns: Retry, Circuit Breaker, Timeouts

Resilience Patterns: Retry, Circuit Breaker, Timeouts

Communication & Integration Patterns

Mastering Software Architecture Patterns and System Design

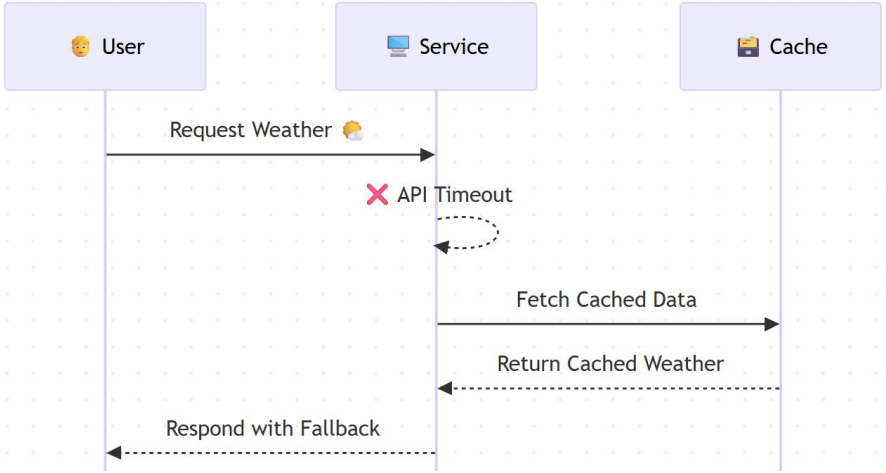
Why Resilience Matters

- Modern systems are distributed, interconnected, and fail in unpredictable ways
- Resilience patterns allow systems to handle failure gracefully without cascading breakdown
- Goal: degrade, don’t collapse



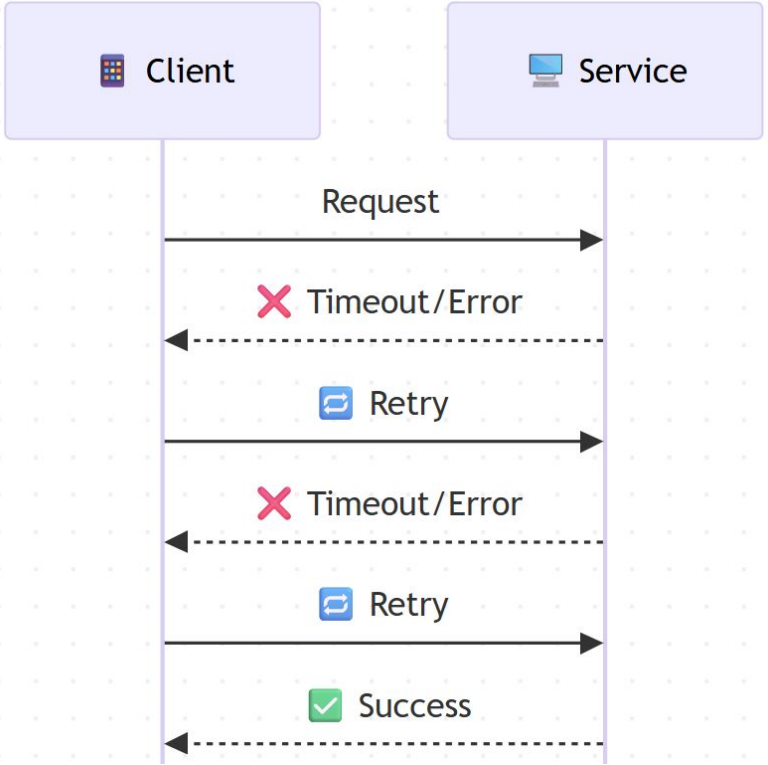
Graceful Failure

- Instead of crashing, return:
  - Default response
  - Cached result
  - Fallback handler
- Prevents domino effects in failure scenarios
- Example: Show weather from cache if API is down



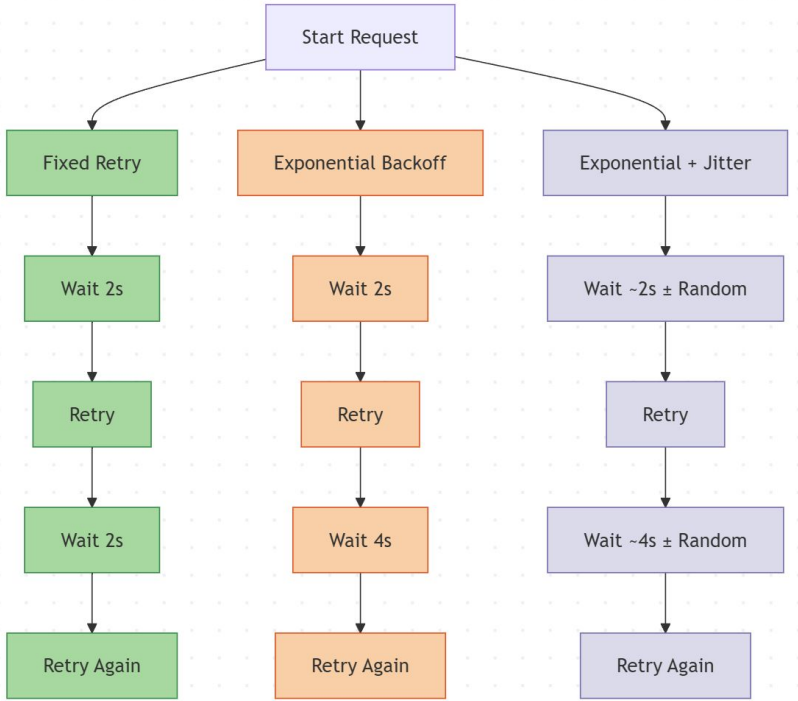
Retry Pattern

- Retry a failed operation after short delay
- Useful for transient errors: network hiccups, temporary outages
- Risks: too many retries → overload
- Best with: limits + delays



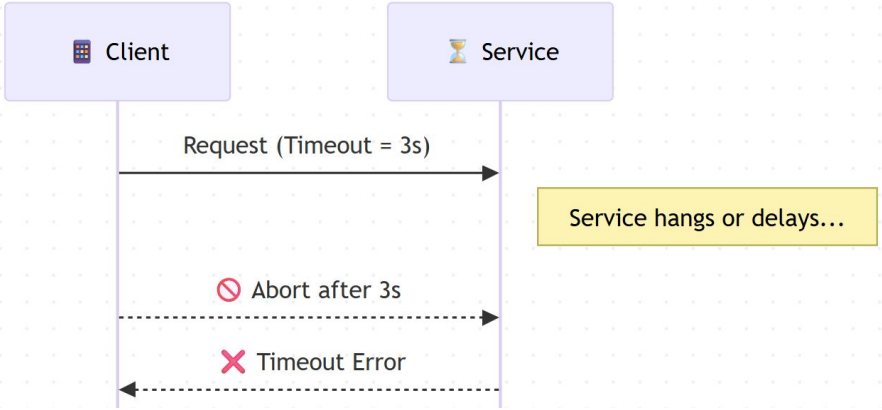
Retry Strategies

- Fixed Retry: retry after fixed delay (e.g. every 2s)
- Exponential Backoff: increase delay between retries
- Jitter: add randomness to avoid synchronized retry storms



Timeout Pattern

- Don’t wait forever — define how long to wait before giving up
- Protects threads and system resources
- Must balance between:
  - Too short → false failures
  - Too long → blocks system



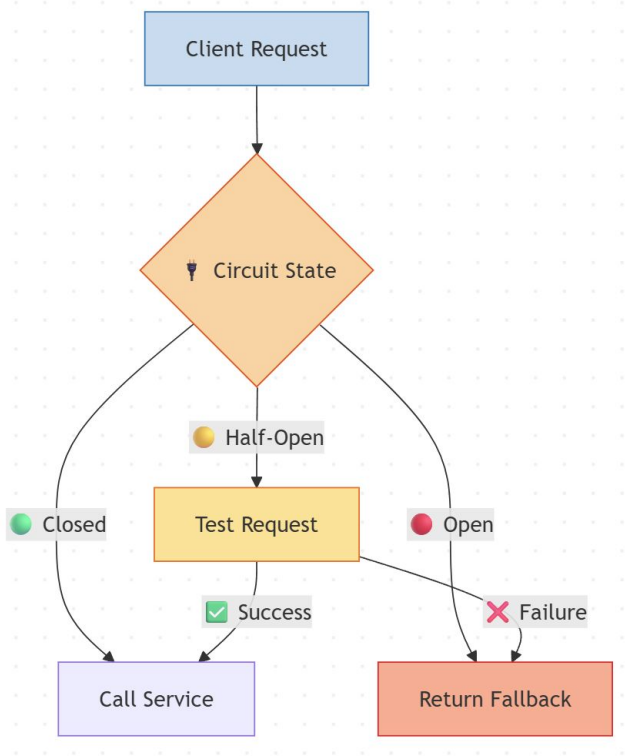
Circuit Breaker Pattern

- Like a fuse in electronics
- Tracks error rates & opens the circuit if failures exceed threshold
- Three states:
  - Closed: normal
  - Open: reject requests
  - Half-Open: test one request before resuming



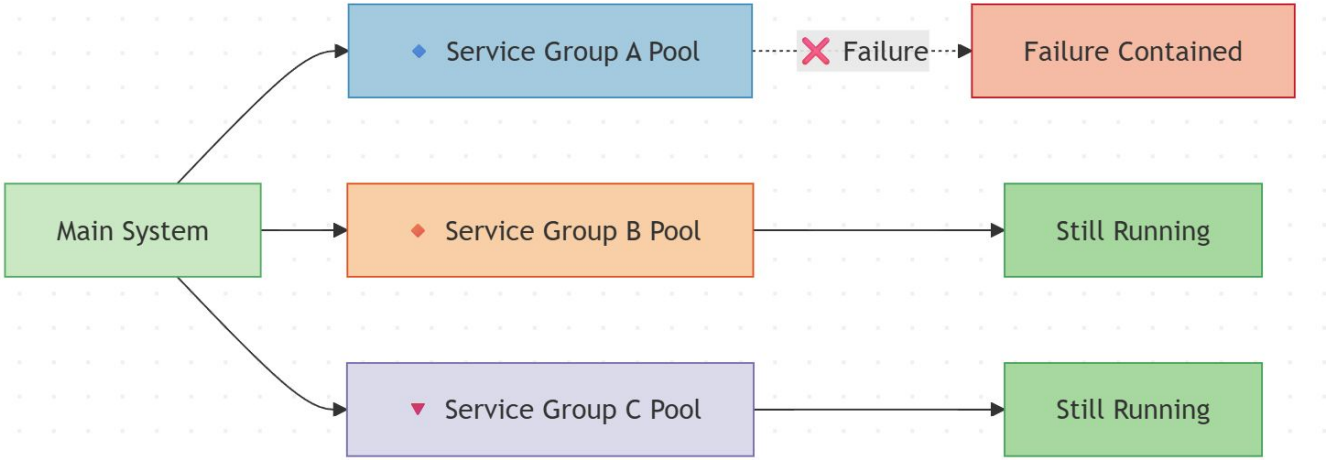
Circuit Breaker in Action

- Prevents overloading a failing service
- Redirects traffic or returns fallback
- Helps system recover before retrying
- Example: payment service fails → circuit opens → fallback message shown



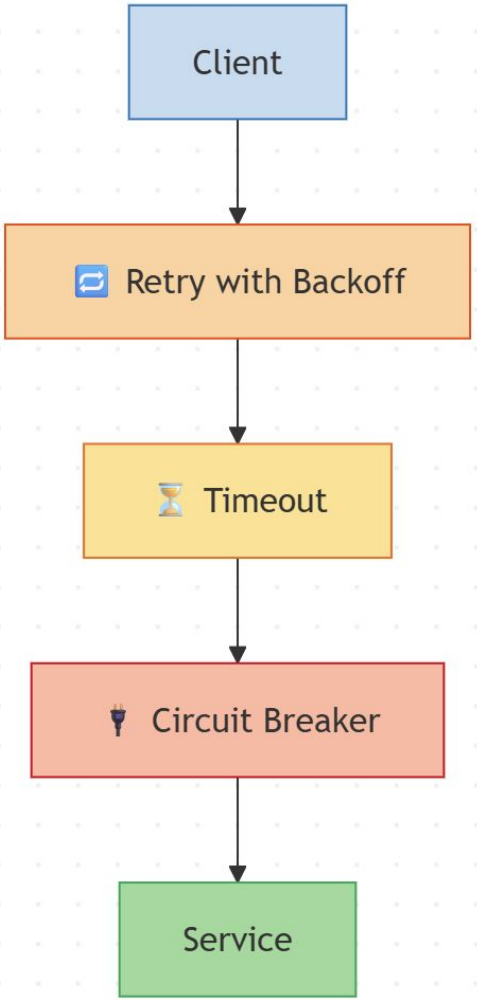
Bulkhead Pattern

- Isolate resources into independent compartments
- Prevents one component's failure from affecting others
- Analogy: ship with watertight sections
- Use cases: thread pools, queues per service group



Combining Resilience Patterns

- Real systems often combine patterns:
  - Retry + Timeout
  - Circuit Breaker + Fallback
  - Bulkhead + Retry
- Each layer adds safety against failure modes



Design Considerations

- Don’t overuse retries → cascading overload
- Tune timeout per service
- Monitor circuit breaker metrics
- Use observability tools (traces, logs, metrics)
- Test failure modes (chaos testing)

Summary - Resilience Patterns: Retry, Circuit Breaker, Timeouts

- Resilience patterns make systems fault-tolerant, not fault-proof
- Balance between availability and performance
- Key tools in modern architecture toolbox
- Next: Apply these in real-world design scenarios
- What’s next:
  - Building Fault-Tolerant Systems

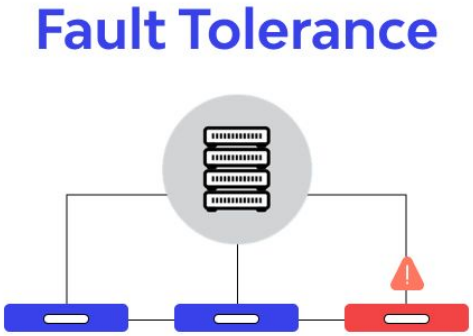
Building Fault-Tolerant Systems

Communication & Integration Patterns

Mastering Software Architecture Patterns and System Design

Why Fault Tolerance Matters

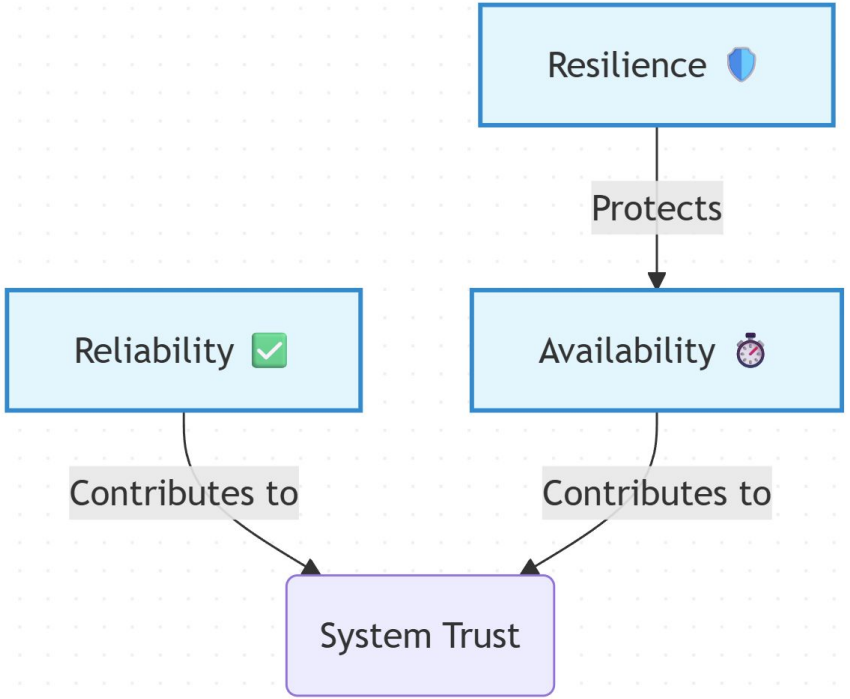
- Business impact: downtime ⇒ lost revenue, trust, SLA penalties.
- Technical reality: Mean-time-between-failure (MTBF) is shrinking as systems grow.
- Goal: Keep user-visible availability high even when individual components are sick.



Reliability, Availability & Resilience

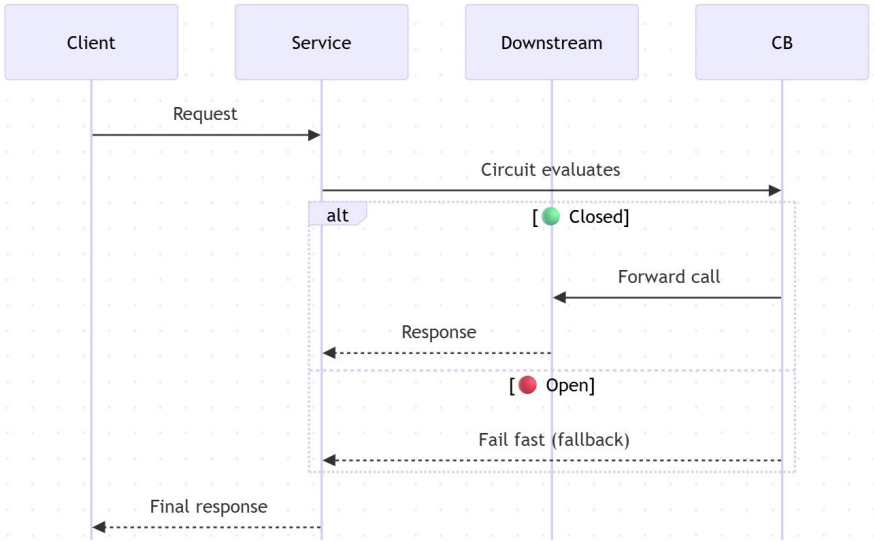
- Reliability: probability your system does the right thing for a period.
- Availability: percentage of time the system responds at all.
- Resilience: system's ability to recover quickly when something breaks.

Think of reliability as "no wrong answers," availability as "always answers," and resilience as "bounces back when punched."



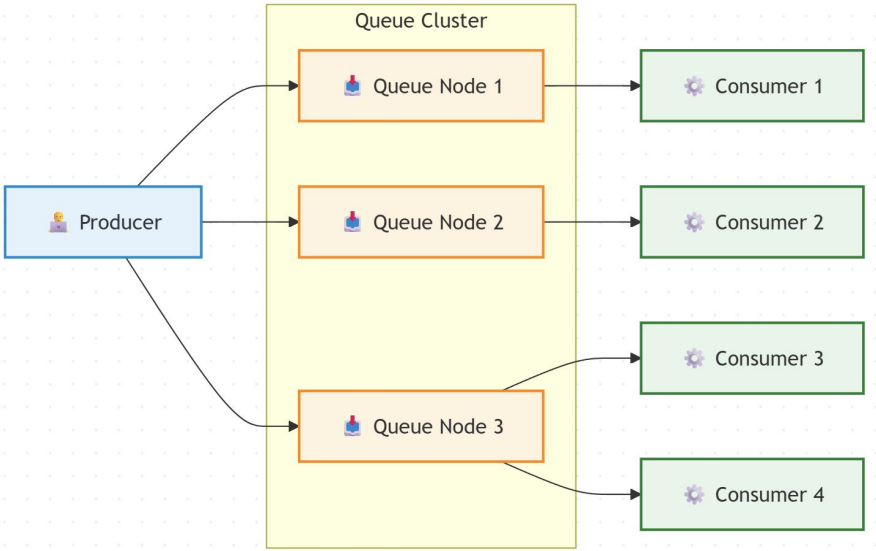
Core Resilience Patterns

- Circuit Breaker: Stop sending traffic to a broken dependency.
- Timeouts: Don’t wait forever; free up threads.
- Retries with Jitter: Retry few & fast, then back off.
- Bulkheads: Isolate workloads to prevent flood-the-boat failures.



Messaging for Resilience

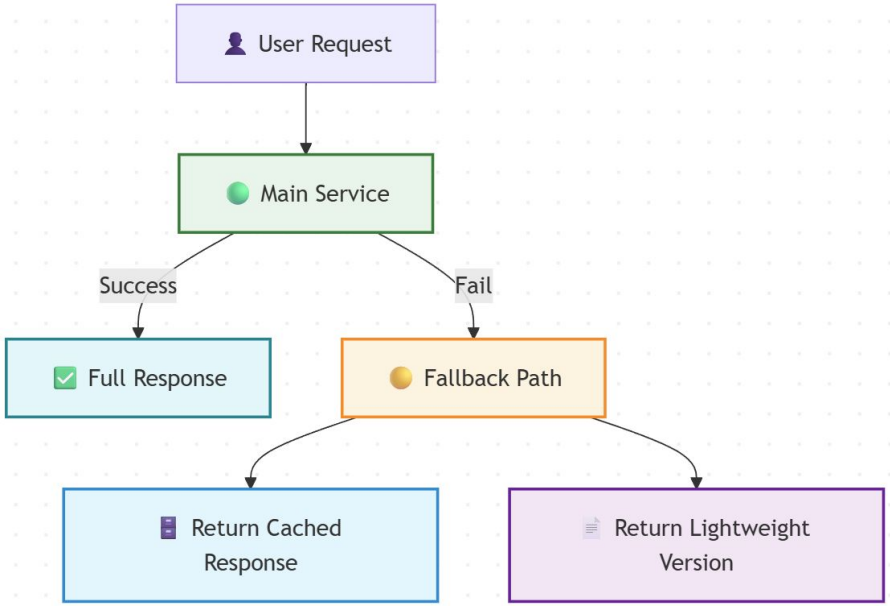
- Queues & Streams decouple producers/consumers ➜ absorb spikes.
- At-least-once delivery + idempotent handlers ⇒ safe retries.
- Ordering vs. availability: partitions give you isolation when a shard stalls.



Fallback Strategies

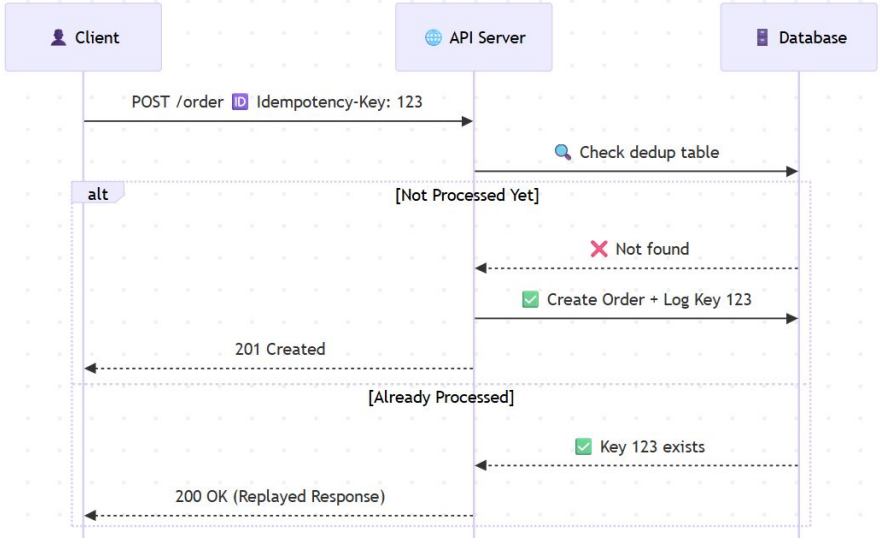
- Graceful degradation: serve low-fidelity response (e.g., cached product list).
- Static defaults: show “sorry, Ratings unavailable ★★★☆☆” instead of 500 error.
- Shadow features: toggle off non-critical modules to protect core path.

Users prefer stale data over error pages — design with that trade-off in mind.



Designing for Idempotency

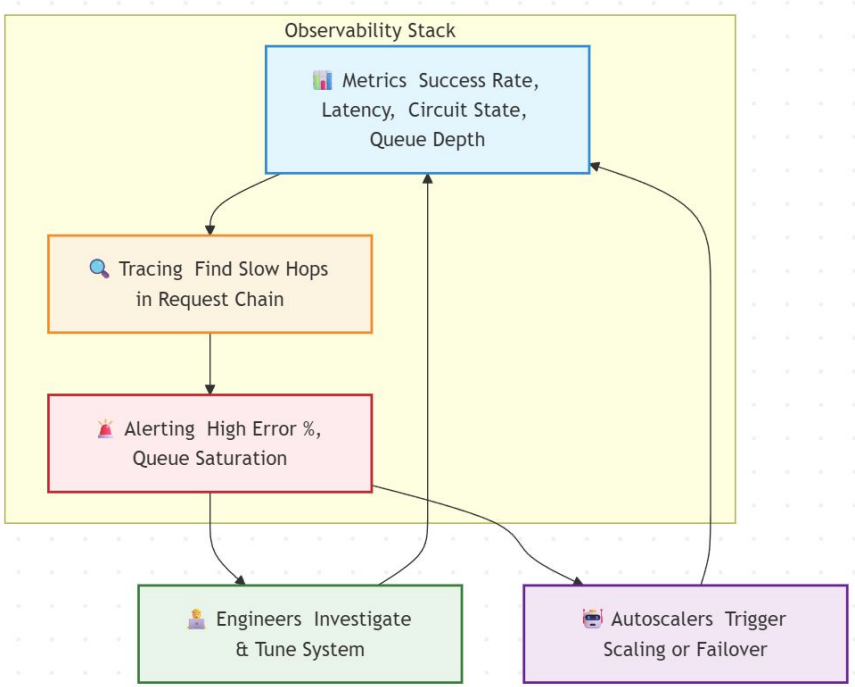
- Rule: same message, same effect, ∞ times.
- Techniques
  - Unique idempotency keys in HTTP headers
  - Dedup tables with TTL
  - Versioned events (v2.OrderCreated)
- Result: safe to repeat when network uncertainty strikes.



Observability & Feedback Loops

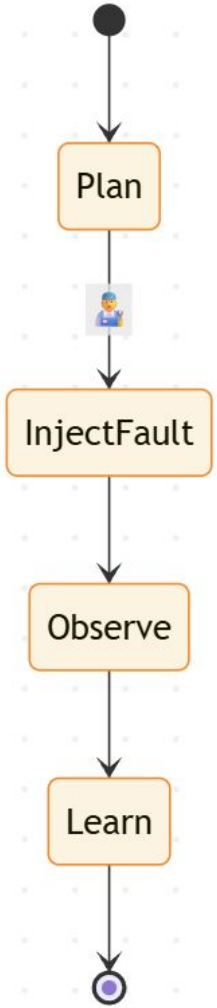
- Metrics: success rate, latency, circuit state, queue depth.
- Tracing: pinpoint slow hops in a request chain.
- Alerting: symptom-oriented (↑ error %, ↑ saturation).

You can’t fix what you can’t see — feed data back to engineers and autoscalers.



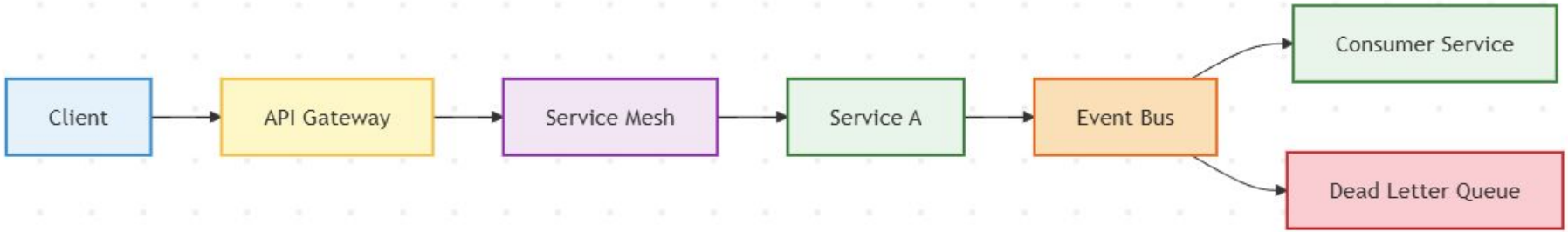
Chaos Engineering Basics

- Hypothesis-driven experiments: “If service-B dies, checkout still works.”
- Blast radius control: start small, ramp up (📈).
- Tooling: Chaos Monkey, Litmus, Azure Chaos Studio.
- Metrics to watch: user errors, latency, fallback activation.



Bringing It All Together

- Pattern mash-up:
  - Client → API Gateway with timeouts & retries
  - Gateway → Service mesh (circuit breakers)
  - Services communicate via event bus with dead-letter queues
  - Continuous chaos tests in staging ➜ confident releases
- Key takeaway: Resilience is an ecosystem, not a single plug-in.



Summary - Building Fault-Tolerant Systems

- Failures are inevitable in distributed systems — resilience is not optional.
- Use timeouts, circuit breakers, bulkheads, and fallbacks to localize and contain failures.
- Message queues and asynchronous processing absorb spikes and decouple services.
- Idempotency techniques prevent duplicate side effects during retries.
- Chaos engineering helps validate real-world resilience before failure strikes.
- What’s next:
    - Security & Governance in Architecture